

Sicherheitsprüfung

Statische Codeanalyse und Prüfung der Abhängigkeiten

SIMS – Security Incident Management System

Team	CMSLPD
Stand	13. November 2026
Branch	code-cleared
Umfang	Backend (.NET 10) und Frontend (React/TypeScript)

1 Zusammenfassung

Die Codebasis von SIMS wurde am 13.11.2026 mit einer statischen Codeanalyse (SAST) und einer Prüfung der eingebundenen Bibliotheken auf bekannte Schwachstellen untersucht. Im eigenen Quellcode und in den .NET-Paketen wurden **keine Schwachstellen** gefunden. Im Frontend meldet npm zwei Schwachstellen mittleren Schweregrads in der Bibliothek react-router. Beide sind in der Konfiguration von SIMS **nicht ausnutzbar**, sollten aber durch ein Update behoben werden.

0	0	2
Funde im Quellcode (Semgrep, 250 Dateien)	anfällige .NET-Pakete (11 Projekte)	mittlere Befunde npm (nicht ausnutzbar)

Bereich	Werkzeug	Ergebnis	Bewertung
Eigener Quellcode (C#, TypeScript)	Semgrep	0 Funde	**unauffällig**
NuGet-Pakete inkl. transitiver	dotnet list package --vulnerable	0 anfällige Pakete	**unauffällig**
npm-Pakete (Produktion)	npm audit --omit=dev	2 x mittel (react-router)	**Update empfohlen**

2 Umfang und Vorgehen

Geprüft wurde der Stand der Branch code-cleared im Repository ac_ProjectSeks. Die Prüfung umfasst alle Dienste des Systems:

Komponente	Technologie	Aufgabe
AuthApi	.NET 10, ASP.NET Core Identity	Anmeldung, JWT, MFA, Benutzer- und Teamverwaltung
ScanApi	.NET 10, EF Core, SQL Server	Projekte, SBOM-Upload, Scans, Tickets, Audit
NewsApi	.NET 10	Öffentliche Sicherheitsmeldungen aus RSS-Feeds
Worker	.NET 10 Hintergrunddienst	Abgleich mit OSV, KEV und EPSS, Urteil, Eskalation
Web	React 18, TypeScript, Vite	Weboberfläche

Eingesetzte Verfahren

Verfahren	Werkzeug	Was geprüft wird
Statische Codeanalyse (SAST)	Semgrep (Docker-Image semgrep/semgrep), Regelsätze p/csharp, p/typescript, p/react, p/secrets	Unsichere Muster im eigenen Code, z.B. Injection, unsichere Kryptografie, XSS, hartcodierte Geheimnisse
Abhängigkeiten Backend (SCA)	dotnet list package --vulnerable --include-transitive	Bekannte Schwachstellen in direkt und indirekt genutzten NuGet-Paketen
Abhängigkeiten Frontend (SCA)	npm audit --omit=dev	Bekannte Schwachstellen in npm-Paketen, die im ausgelieferten Frontend landen

3 Ergebnisse

3.1 Statische Codeanalyse (Semgrep)

Semgrep hat alle von Git verwalteten Dateien geprüft. 26 Dateien wurden über die Standard-Ausschlussliste (.semgrepignore) übersprungen, etwa Build-Ausgaben und Abhängigkeiten. Rund 99,7 % der Zeilen konnten vollständig analysiert werden.

Sprache	Regeln	Dateien
C#	27	139
TypeScript	74	40
JavaScript	68	2
YAML	1	2
Sprachübergreifend (u.a. Secrets)	36	250
Gesamt	**138 ausgeführt**	**250**

Ergebnis: 0 Funde, davon 0 blockierend.

3.2 NuGet-Pakete (Backend)

Für alle 11 Projekte der Solution (7 Anwendungs- und Bibliotheksprojekte, 4 Testprojekte) liegen laut nuget.org keine anfälligen Pakete vor, auch nicht unter den transitiven Abhängigkeiten.

Ergebnis: 0 anfällige Pakete.

3.3 npm-Pakete (Frontend)

npm audit meldet zwei Schwachstellen mittleren Schweregrads im Paket react-router, das über react-router-dom (Version 6.x, deklariert als ^6.28.0) eingebunden ist. Behoben sind beide erst ab Version 7.18.0.

Kennung	Beschreibung	Schwere	Betroffen	Behoben
CVE-2026-53669 GHSA-wrjc-x8rr-h8h6	Open Redirect über Backslash in <Link> und useNavigate	Mittel CVSS 4.0: 5.1	6.0.0 – 7.17.0	7.18.0
CVE-2026-53666 GHSA-337j-9hxr-rhxg	Constructor Injection über deserializeErrors() bei SSR-Hydration	Mittel CVSS 3.1: 6.1	6.4.0 – 7.17.0	7.18.0

4 Bewertung der npm-Befunde

Beide Schwachstellen setzen Voraussetzungen voraus, die in SIMS nicht gegeben sind:

Befund	Voraussetzung für einen Angriff	Situation in SIMS	Ausnutzbar
CVE-2026-53669 Open Redirect	Ein Angreifer kann den Zielpfad von <Link> oder useNavigate beeinflussen, z.B. über einen Rücksprung-Parameter in der URL.	Alle Navigationsziele sind feste Pfade im Code (z.B. /dashboard, /mfa). Es gibt keinen Rücksprung-Parameter aus der URL.	**Nein**
CVE-2026-53666 Constructor Injection	Die Anwendung nutzt Framework Mode oder Data Mode mit serverseitigem Rendering (SSR) und Hydration.	SIMS nutzt <BrowserRouter> im Declarative Mode und rendert ausschließlich im Browser, ohne SSR.	**Nein**

Das tatsächliche Risiko ist damit gering. Da ein Update auf react-router 7.18.0 für eine Anwendung im Declarative Mode mit geringem Aufwand möglich ist, wird es trotzdem empfohlen. So bleibt der

Bericht von npm audit sauber und spätere Codeänderungen, etwa ein Rücksprung nach dem Login, führen nicht unbemerkt eine Lücke ein.

5 Maßnahmen aus dem Code-Review

Vor dieser Prüfung wurde die Codebasis manuell durchgesehen. Die dabei gefundenen sicherheitsrelevanten Punkte sind auf der Branch code-cleared bereits behoben:

Bereich	Problem	Maßnahme
Scan-API: Zugriffsschutz	Scans konnten unabhängig vom Team des Projekts hochgeladen, aufgelistet und gelesen werden.	Upload, Liste und Detailansicht prüfen jetzt das Team des Projekts. Admins sehen weiterhin alles.
Scan-API: Konfiguration	Ohne Konfiguration startete die API mit einem fest hinterlegten JWT-Schlüssel und einer Verbindungszeichenfolge mit Passwort.	Fehlen JWT-Schlüssel oder Verbindungszeichenfolge, startet die API nicht.
Scan-API: CORS und Rate Limit	CORS war fest auf localhost gesetzt. Das Limit von 100 Anfragen pro Minute galt für alle Nutzer gemeinsam.	CORS-Ursprünge kommen aus der Konfiguration, das Rate Limit gilt pro IP-Adresse.
Scan-API: SBOM-Upload	Ein SBOM mit mehreren Fehlern führte zu einem Serverfehler (HTTP 500).	Alle Fehler werden gesammelt als Validierungsfehler (HTTP 400) zurückgegeben.

6 Empfehlungen

- **react-router aktualisieren:** react-router-dom auf mindestens 7.18.0 anheben, danach TypeScript-Prüfung und einen kurzen Durchlauf durch Anmeldung und Navigation.
- **Prüfung regelmäßig wiederholen:** Semgrep, dotnet list package und npm audit vor jeder Abgabe bzw. jedem Release ausführen, idealerweise automatisiert in einer CI-Pipeline.
- **Eigenes Werkzeug nutzen:** Für SIMS selbst eine SBOM erzeugen und in SIMS hochladen, damit neue Schwachstellen in den Abhängigkeiten laufend erkannt und als Ticket nachverfolgt werden.

Anhang: Ausgeführte Befehle

```
cd C:\00Studium\ac_ProjectSeks
docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep scan `
  --config p/csharp --config p/typescript --config p/react --config p/secrets `
  --exclude node_modules --exclude bin --exclude obj --exclude dist

dotnet list sims.slnx package --vulnerable --include-transitive

cd src\Web
npm audit --omit=dev
```